PJSIP SDK Integration Reference Guide

Objective: Quick help guide will help developer to get start with the SDK integration steps.

SDK Available:

- iOS: Apple iPhone, iPad, and iPod Touch
- Android
- Windows Mobile
- Windows Phone 8.x and UWP
- SDK Download link https://www.pjsip.org/

IDE/Software Requirement:

Besides the Android SDK, you will also need the Android NDK.

- Optional if you want to build and and run the sample applications (i.e: pjsua2 and pjsua):
- SWIG
- Eclipse with ADT Plugin (deprecated since 2.6)
- Android Studio
- Telnet application to interact with PJSUA command line. If you are not familiar with telnet, please find a tutorial suitable for your development platform.

* Activate Android specific settings in the 'config_site_sample.h'

```
#define PJ_CONFIG_ANDROID 1
#include <pj/config_site_sample.h>
```

Getting Started: Building for Android

1. Downalod SDK for desried environemnt like android.
2. Besides the Android SDK, you will also need the Android NDK.
3. Include following in config file
4. Install SWIG compilier
5. Use SWIG command. SWIG is a compiler that takes C/C++ declarations and creates the wrappers needed to access those declarations from other languages including including Perl, Python, Tcl, Ruby, Guile, and Javas
6. Run code using

```
$ cd /path/to/your/pjsip/dir
$ export ANDROID_NDK_ROOT=/path_to_android_ndk_dir
$ ./configure-android
$ make dep && make clean && make
```

Event Control Methods:

SIP Registration:

```
void sipReg() {

	String s1 = "sip:100003622@11.111.111.111:5060";
	String s2 = "sip:11.111.111.225:5060";
	String acc_id = s1;
	String registrar = s2;
	String proxy = "";
	String username = "100003622";
	String sipPassword = "1234567";
	accCfg.setIdUri("sip:100003622@11.111.111.225:5060");
	accCfg.getRegConfig().setRegistrarUri(registrar);
	AuthCredInfoVector creds = accCfg.getSipConfig().
			getAuthCreds();
	creds.clear();
	if (username.length() != 0) {
		creds.add(new AuthCredInfo("Digest", "*", username, 0,
				sipPassword));
	}
	StringVector proxies = accCfg.getSipConfig().getProxies();
	proxies.clear();
	if (proxy.length() != 0) {
		proxies.add(proxy);
	}
	/* Enable ICE */
	accCfg.getNatConfig().setIceEnabled(true);
	/* Finally */
	lastRegStatus = "";
	try {
		account.modify(accCfg);
	} catch (Exception e) {
	}
}
```

Make Call:

```
public void makeCall(String buddy_uri) {

	if (currentCall != null) {
		return;
	}

	MyCall call = new MyCall(account, 1);
	CallOpParam prm = new CallOpParam(true);
	try {
		call.makeCall(buddy_uri,  prm);
	} catch (Exception e) {
		call.delete();
		return;
	}
	currentCall = call;
	showCallActivity();
}
```

Hold Call:

```
public void holdCall(View view) {
	CallOpParam prm = new CallOpParam(true);
	try {
		currentCall.setHold(prm);
		buttonConference.setVisibility(View.GONE);
		buttonUnhold.setVisibility(View.VISIBLE);
		buttonHangup.setVisibility(View.VISIBLE);
	} catch (Exception e) {
		e.printStackTrace();
	}

	view.setVisibility(View.GONE);
}
```

Hang Up Call:

```
public void hangupCall(View view) {
	handler_ = null;
	finish();
	if (currentCall != null) {
		CallOpParam prm = new CallOpParam();
		prm.setStatusCode(pjsip_status_code.PJSIP_SC_DECLINE);
		try {
		currentCall.hangup(prm);
		} catch (Exception e) {
			System.out.println(e);
		}
	}
}
```

Transfer call:

Step 1: implement bellow class for using transfer method

Import static org.pjsip.pjsua2.app.Main.*currentCall*

Step 2: Write a transfer method and call *currentCall*.xfer(transferString,**prm**) method of library to initiate the transfer event for desired number.

Below is the sample code for transfering a call

```
public void transferTo(String destination) throws Exception {
   String transferString;
     if (destination.startsWith("sip:")) {
          transferString = "<" + destination + ">";
     } else {
          if ("*".equals("sip:100003669@11.111.111.225:5060")) {
               transferString = "<sip:" + destination + ">";
          } else {
               transferString = "<sip:" + destination + "@" + "sip:100003669@11.111.111.225:5060" +
">";
          }
     }
```

```
// CallOpParam prm = new CallOpParam(true);
currentCall.xfer(transferString,prm);
```

Click To Call API Implementation :

Click To Call API – Outbound Dialing

Once tenant and extension authentication is completed the call system will originate call to establish communication between the caller (extension) and callee (phone number).

URL: https://crm2.cloud-connect.in/ccpl_api/v1.4/api/info/click2call
Method: Post
Content -Type: Application/JSON

Required Parameters:

Input Parameters	Description
token_id (mandatory)	Token ID for the authentication
extension_number (mandatory)	Extension Number on which Call will first come
extension_password (mandatory)	Web Password of ext Number for authentication in MD5 encryption
phone_number (mandatory)	Phone Number on which Call will be made once extension answers the call

Request:

```
{
        "token_id" : "pP9HXXXXXXXX",
        "phone_number" : "1234567890"
        "extension_number" : "101",
        "extension_password" : "7367cc4cee061amfkfjfgf89gggjkg",
}
```

Response:

```
{
    "status": "SUCCESS",
    "message": "Call Dialled Successfully to Extension.",
    "data": "5e184ad3-0c3e-408a-be83-55ccb06eb256",
    "unique_token": "a069af42-c569-4887-96cf-
    2957bcb481f8","code": "200",
    "status_reason": "OK"
}
```

Possible Responses

Status Code	Status Message	Reason
200	OK	When Call Dialed Successfully to Extension
424	Method Failed	When Invalid URL is called
406	Not Acceptable	When Invalid Parameter is passed. Which parameter is having problem will be given back in message field
500	Failed to Dial	When System is not able to dial extension phone number or extension not answered

Call Log or Call Details API :

CloudConnect will receive API call to get the call log details.

URL: https://crm2.cloud-connect.in/ccpl_api/v1.4/api/info/{startDate}/{endDate}/{userType}/callLog

Method: Post

Content -Type: Application/JSON

Required Parameters:

Input Parameters	Description
token_id (mandatory)	Token ID for the authentication
unique_token(optional)	unique_token received in Click2Call API will assist to find only that particular call record with details.
startdate (mandatory)	From date to filter the CDR data.
enddate(mandatory)	To date to filter the CDR data.
userType(mandatory)	Tenant as static value

Request:

{
"token_id":"pP9HzIXDFUXubbmi",
"unique_token": "f453d2e0-b30f-4f81-9906-fac13a1988e7"
}

Response:

{
"status": "SUCCESS",
"message": "Call Log Details.","data":[
{
"callid": "501608",
"tenant_id": "100003",
"start_date": "2020-06-18 16:27:00",
"answer_time": "2020-06-18 16:27:09",
"end_date": "2020-06-18 16:27:32",
"caller": 620",
"callee":
"7827XXXXXX"
,
"forward": "+917827XXXXXX",
"call_direction":
OUTBOUND","did":
"N/A",
"call_sec": "32",
"call_minute": "00:32",
"answer_sec": "23",
"answer_minute": "00:23",
"call_status": "Answered",
"hangup_by": "CUSTOMER",
"hangup_reason":

```
            "NORMAL_CLEARING",
            "incoming_call_charges":
            "0.00000",
            "outgoing_call_charges": "0.00000",
            "call_rec_path": "https://crm2.cloud-
            connect.in/XXX/fs_XXX/recordings/0/100XXX/auto_rec/2020-06-18/7e8d5e66-e5dc-
            475a-a874-8d65b92a0c7e.wav"
            }
          ]
}
```


Possible Responses

Status Code	Status Message	Reason
200	OK	When Call Log Details successfully fetched. In JSON response the data which user will get are:- tenant_id,start_date,end_date,caller,callee,forward.
404	Missing Parameter	When mandatory parameters are missing.
405	No Token	When Token ID parameter is missing.
406	Tenant Not Active	When Tenant is Inactive or Deleted.
408	No Tenant Found	When Requested Tenant data not found.
411	No Call Log Details Found	When Requested Call Log Details not found.
412	Invalid Start Date	When Start Date is in invalid format. Allowed format is : 'YYYY-MM-DD'
413	Invalid End Date	When End Date is in invalid format. Allowed format is : 'YYYY-MM-DD'
414	Invalid User Type	When User Type is invalid. Allowed Type is: 'TENANT'
424	Method Failed	When Invalid URL is called
500	Internal Server Error	When any Internal Server Error is occurred.
501	Authentication Failed	When passed Token is not match with valid user type or user does not have API access.
502	Exception Thrown	When any Exception is thrown.

Real time Callback/Webhook Events

Parameter Name	Description
Uuid	Unique ID of call
extension_number	Extension number which is ringing (This number will be the one which is used for registration.)
caller_name	URL encodeed caller name who is calling the extension
Caller_number	URL encoded caller number who is calling the extension

call_status	Status of the call. As of now this will always give “Ring”

Example Callback URL and Webhook Events

CCPL will trigger 3 events for call RING, ANSWER and Hang-up

POST(in JSON) :

unique_token=077ac226-ae56-410c-aa1b-45f4d527eb6b
uuid=3a0b05f0-ed98-4644-ab99-d1ae86c952a6
extension_number=100928703
caller_name=7827966681
caller_number=7827966681
call_status=Ring
call_direction=outbound

POST(in JSON) :

unique_token=077ac226-ae56-410c-aa1b-45f4d527eb6b
uuid=3a0b05f0-ed98-4644-ab99-d1ae86c952a6
extension_number=100928703
caller_name=7827966681
caller_number=7827966681
call_status=Answer
call_direction=outbound

POST(in JSON) :

unique_token=077ac226-ae56-410c-aa1b-45f4d527eb6b
uuid=3a0b05f0-ed98-4644-ab99-d1ae86c952a6
extension_number=100928703
caller_name=7827966681
caller_number=7827966681
call_status=Hangup
call_direction=outbound
call_answer_time=2023-02-28%2011%3A56%3A33
call_duration=14
reason=NORMAL_CLEARING

**********END**********